原理

记$\mathrm{dim_z} = N$，为模型宽度。构造一组初始化、学习率与特殊缩放组合，满足三个原则：

- 原则1：任何神经网络中单元素计算结果均为$\Theta(1)$。如输入层$x = \Theta(1)$，中间隐藏层$y = Wx = \Theta(1)$。即：向量x的每个元素、向量y的每个元素大小数量级均为$\Theta(1)$。后续称之为**坐标为$\Theta(1)$**。根据正态分布的性质，单元素大小为$\Theta(1)$对应单元素方差为$\Theta(1)$
- 原则2：模型最终输出的结果（即每个位置下，对每个词的权重score）为$O(1)$，即：至少不随模型宽度增加而增长，使得输出层score不可能爆炸
- 原则3：参数做一轮更新后，保证所有神经网络中单元素计算结果的变化量都为$\Theta(1)$，即$\Delta y = \Delta W x, y_i = \Theta(1)$，这也可使得即使在训练初期，条件2对应的输出层score可能为$o(1)$，也可最终变化为$\Theta(1)$

构造方法

一、可学习参数归类

1. **Input Weights**
   定义：记参数规模为$R^{a*b}, a = \Theta(1),\ \ b = \Theta(N)$则为input weights。所以biases也分类为input weights
   对应代码中的变量 (Python Path)：
   - `model.unary_factors.weight` (Embedding)
   - `cls.predictions.bias` (Decoder Bias, 视作 Input 类处理)
   - `cls.predictions.transform.dense.bias` (Decoder Bias, 视作 Input 类处理)
   - `cls.predictions.transform.LayerNorm.bias` (Decoder Bias, 视作 Input 类处理)
   - `cls.predictions.transform.LayerNorm.weight`
2. **Hidden Weights**
   定义：记参数规模为$R^{a*b}, a = \Theta(N),\ \ b = \Theta(N)$则为hidden weights
   对应代码中的变量 (Python Path)：
   - `model.iterator.head_selection.ternary_factor_u`
   - `model.iterator.head_selection.ternary_factor_v`
   - `model.iterator.topic_modeling.binary_factor`
   - `cls.predictions.transform.dense.weight` (MLM Head 内部的线性层)
3. **Output Weights**
   定义：记参数规模为$R^{a*b}, a = \Theta(N),\ \ b = \Theta(1)$则为output weights

对应代码中的变量 (Python Path)：

- `cls.predictions.decoder.weight` (映射回 `vocab_size`)

4. **Scalars**
   定义：标量乘子 (需为常数)
   对应代码中的变量 (Python Path)：
   - `ternary_factor_scaling` , `binary_factor_scaling` , `classifier_amplifier` , `regularize_h/z/g`

二、参数化方法

假设基础学习率为 η。这个基础学习率是一个需要调参的常数超参。

参数类别	初始化分布(均值与方差)	学习率设置 (Optimizer)
除**Biases**外的**Input Weights**	$\mathcal{N}(0,1)$	**Base LR (η)**
Hidden Weights	$\mathcal{N}(0,\frac{1}{N})$	**Scaled LR ($\frac{\eta}{N}$)**
Output Weights	$\mathcal{N}(0,\frac{1}{N^2})$	**Scaled LR ($\frac{\eta}{N}$)**
Biases	常数 0	**Base LR (η)**

- 关于模型架构，dim_g, ternary_rank均与dim_z成倍线性缩放

三、特殊缩放

1. 归一化层
   L1 归一化会导致每个元素的值变为 $O(1/N)$，导致信号坍塌。（具体原因见后续推导）

- **影响位置**： `SquaredSoftmax` 和 `AbsNormalization` 的 `forward` 函数。（注：在先前调参中已确定使用的归一化方法为： `potential_func_z=square` , `potential_func_z=abs` ）
- **修改方法**：归一化后再额外乘上N

```
# 原代码
hidden_states = F.normalize(hidden_states, p=1, dim=self.dim, eps=self.eps)
# 修改后 (增加 * self.dim 缩放)
hidden_states = F.normalize(hidden_states, p=1, dim=self.dim, eps=self.eps) * hidden_states
```

2. HeadSelection
   HeadSelection中的u, v点积操作如果不缩放，数值会随 rank 增加而线性增加。（具体原因见后续推导）

- **影响位置**： `PtHeadSelection` 中计算 `message_F` 的地方。

- **修改方法**： head selection计算结果额外除以$\Theta(N)$（即ternary_rank）

```
message_F = torch.matmul(qz_u, qz_v.transpose(2, 3))
# 修改后 (添加一行，额外除以秩)
message_F = message_F / self.ternary_rank
```

3. `cls` 头的重新初始化
   手动定位到 `self.cls.predictions.decoder` 和 `self.cls.predictions.transform.dense` 并按照第 2 点表格中的规则强制 `data.normal_()` ，不遵循默认的标准初始化方法。

推导

原则一推导：任何神经网络中单元素计算结果均为 $\Theta(1)$

1. 隐藏层线性变换

包括 Topic Modeling 和 Head Selection 中的投影层，以及 BERT 头的 dense 层。

- **计算形式**：$y = Wx$
- **配置**：Hidden Weight 初始化方差 $\mathrm{Var}(W_{ij}) = 1/N$，输入 x 的坐标 $x_j \sim \Theta(1)$。
- **推导**：
  输出向量 y 的第 i 个分量为：

$$y_i = \sum_{j=1}^{n} W_{ij} x_j$$

  假设 W 与 x 独立，计算方差：

$$\mathrm{Var}(y_i) \approx \sum_{j=1}^{n} \mathrm{Var}(W_{ij}) \cdot \mathbb{E}[x_j^2] = n \cdot \frac{1}{n} \cdot \Theta(1) = \Theta(1)$$

2. 特殊缩放：Head Selection

- **计算形式**：$F = (W_u x)(W_v x)^T / \mathrm{rank}$
- **配置**： `message_F = (qz_u @ qz_v.T) / self.ternary_rank` 。
  其中 $\mathrm{qz_u}, \mathrm{qz_v} \in \mathbb{R}^{\mathrm{rank}}$ 来自上一环节的隐藏层变换，故其坐标为 $\Theta(1)$。且 $\mathrm{rank} = \Theta(n)$。
- **推导**：
  外积矩阵未缩放前的元素 $(qz_u \cdot qz_v^T)_{ij}$：

$$\text{Raw}_{ij} = \sum_{k=1}^{\text{rank}} (qz_u)_{ik}(qz_v)_{jk}$$

这是 rank 个 $\Theta(1)$ 随机变量之和。根据中心极限定理或大数定律，其数值量级（方差或期望）为 $\Theta(\text{rank}) = \Theta(N)$。
应用缩放操作：

$$F_{ij} = \frac{\text{Raw}_{ij}}{\text{ternary_rank}} = \frac{\Theta(N)}{\Theta(N)} = \Theta(1)$$

3. 特殊缩放：归一化层

- **计算形式**：$\text{L1Norm}(z) \times N$
- **配置**： `F.normalize(p=1) * dim` 。
  假设输入 z 满足 $z_i \sim \Theta(1)$（例如经过平方处理后，$E[z^2]$ 仍为常数）。
- **推导**：
  计算分母（L1 范数）：

$$S = \sum_{j=1}^{N} |z_j| \approx N \cdot \mathbb{E}[|z|] = \Theta(N)$$

执行归一化后的坐标 $\hat{z}_i$：

$$\hat{z}_i = \frac{z_i}{S} = \frac{\Theta(1)}{\Theta(N)} = \Theta\left(\frac{1}{N}\right)$$

此时如果不加处理，信号已塌缩。执行缩放修正：

$$z'_{\text{final}} = \hat{z}_i \cdot N = \Theta\left(\frac{1}{N}\right) \cdot N = \Theta(1)$$

原则二推导：模型最终输出的结果为 $O(1)$，即至少不随模型宽度增加而增长。

- **计算形式**：$\text{Logit} = W_{out}h$
- **配置**：W_{out} 属于 Output Weights，初始化为 $\mathcal{N}(0, 1/N^2)$。输入 $h \sim \Theta(1)$（由原则一得证）。
- **推导**：
  第 k 个类别的输出分数为：

$$\text{Logit}_k = \sum_{j=1}^{n} (W_{out})_{kj} h_j$$

计算方差：

$$\text{Var}(\text{Logit}_k) = \sum_{j=1}^{n} \text{Var}((W_{out})_{kj}) \cdot \mathbb{E}[h_j^2]$$

代入初始方差 $1/N^2$：

$$\text{Var}(\text{Logit}_k) = N \cdot \frac{1}{N^2} \cdot \Theta(1) = \frac{1}{N}$$

- **具体意义**：
  当 $N \to \infty$ 时，$1/N \to 0$。这意味着在**初始化时**，Logits 的分布极其狭窄，趋近于 0（或一个很小的常数），防止在训练初期即为One-hot分布，导致后续无法有效训练。

原则三推导：一轮更新后，神经网络中元素计算结果的变化量都为 $\Theta(1)$。

使用Adam优化器的条件下，每次可学习参数的更新量大致等于学习率LR, 与梯度绝对大小无关。

1. 隐藏层的最大更新

- **更新公式**：$\Delta h = \Delta W_{hid} x$
- **配置**：Hidden Weight 学习率 $\text{LR} = \eta / N$。
- **推导**：
  对于 Adam，权重更新量 $\Delta W_{ij} \approx \pm \text{LR} = \Theta(1/N)$
  计算激活值的变化量 Δh：

$$\Delta h_i = \sum_{j=1}^{n} \Delta W_{ij} x_j$$

这里有 N 个项相加。每一项是 update ($\approx 1/n$) 乘以 输入 (≈ 1)。
这些 update 不是完全随机抵消的，而是沿着损失函数下降方向产生相干效应（Coherent Update）。根据 Tensor Programs 理论，对齐的总和量级为：

$$\Delta h_i \approx N \cdot \left(\frac{\eta}{N}\right) \cdot \Theta(1) = \Theta(1)$$

2. 输出层的最大更新

- **更新公式**：$\Delta \text{Logit} = \Delta W_{out} h$
- **配置**：Output Weight 学习率 $\text{LR} = \eta / N$。
- **推导**：

$$\Delta W_{out} \sim \Theta(1/N)$$

  Logits 的变化量：

$$\Delta \text{Logit}_k = \sum_{j=1}^{n} \Delta (W_{out})_{kj} h_j$$

  求和项为 N 项。每项量级为 $(1/N) \cdot \Theta(1)$。

$$\Delta \text{Logit}_k \approx N \cdot \frac{1}{n} = \Theta(1)$$

3. 输入层与偏置 (Input & Scalars)

- **配置**：Input Weight 学习率 $\text{LR} = \eta$（常数）。
- **推导**：
  输入层参数不需要考虑线性运算的累加问题，参数自身就是变化量。所以 $\Delta x_i = \text{LR} \times \text{Grad}_{\text{normalized}}$。
  由于 $\text{LR} = \Theta(1)$，输入特征的变化量直接是 $\Theta(1)$。
  偏置 Bias 也是同理（无需考虑累加），$\Delta b = \eta$。

期望结果

对于以下超参：

`learning_rate` `binary_factor_scaling` `ternary_factor_scaling` `classifier_amplifier` `regularize_z` `regularize_h` `regularize_g`

为保持上述推导仍然成立（即三条原则不被违反），数量级都应为$\Theta(1)$。

即：只需一组超参，就可使得不同宽度的模型全部满足三条原则，即数值始终保持稳定，不存在随宽度增加而消失或爆炸的情况。

即：在某个宽度的模型下调好上述超参取值，就可将该组超参直接复制到其他宽度的模型上，并期望同样取得较好的训练结果（训练结果不发散，且loss随模型宽度增大而下降）

验证参数化策略的实验方案

- 在dim_z=256的模型上调参（ternary_rank暂定为dim_z//2, 具体缩放倍率后续还可调整），参数量为17M，取得一组超参组合
- 在更小模型上（dim_z=96，参数量6M），训练4 epochs进行验证，期望模型训练结果不发散
- 在更大模型上（dim_z=1536，参数量142M），训练4 epochs进行验证，期望模型训练结果不发散
- 训练一系列宽度递增的模型（计划训练：96 256 1536 4096），期望模型训练结果全部不发散，且loss随宽度增加而下降（当前256 4096的训练尚未开始，其余两模型已训练）

实验结果及分析

- dim_z=96，训练4 epoch，loss=3
- dim_z=1536（还未训练结束），训练进度epoch=3.51，loss=2.38

为了使三种原则全部满足，以上两模型（宽度相差16倍）充分训练后的可学习参数应满足以下预测情况：

- Input weights(包含Biases)，两模型之间相关参数std的倍率差别应当与宽度倍率差无关
- Hidden weights，两模型之间相关参数std的倍率差别应当接近倍率差开方=4（对应了参数初始化时，初始化标准差数量级为$\Theta(1/\sqrt{N})$）
- Output weights，两模型之间相关参数std的倍率差别应当接近倍率差=16（对应了参数初始化时，初始化标准差数量级为$\Theta(1/N)$）

模型参数输出（dim_z=96）：

```
=== SAVE Step 136928 ===
Last Eval Loss: 2.9680047035217285
Hyperparameters:
dim_z: 96
binary_factor_scaling: 5.615
ternary_factor_scaling: 1.02858
classifier_amplifier: 164.93668
regularize_z: 1.62087
regularize_g: 4.21161
regularize_h: 0.18722
learning_rate: 0.041861


model.unary_factors.weight:
Mean: -5.9336e-01 | Std: 6.8593e+00 | Min: -3.4269e+01 | Max: 3.2649e+01
model.iterator.head_selection.ternary_factor_u:
Mean: -1.4439e-03 | Std: 1.6452e-01 | Min: -8.3068e-01 | Max: 9.7441e-01
model.iterator.head_selection.ternary_factor_v:
Mean: -1.2012e-03 | Std: 1.6802e-01 | Min: -1.0203e+00 | Max: 1.0812e+00
model.iterator.topic_modeling.binary_factor:
Mean: -8.5754e-03 | Std: 1.5814e-01 | Min: -2.0272e+00 | Max: 5.3273e-01
cls.predictions.bias:
Mean: -9.7865e+00 | Std: 4.3324e+00 | Min: -1.6760e+01 | Max: 4.4806e+00
cls.predictions.transform.dense.weight:
Mean: 5.0335e-03 | Std: 1.6292e-01 | Min: -1.1021e+00 | Max: 8.8505e-01
cls.predictions.transform.dense.bias:
Mean: 2.4050e-01 | Std: 1.9100e+00 | Min: -2.7643e+00 | Max: 6.4370e+00
cls.predictions.transform.LayerNorm.weight:
Mean: 8.9494e+00 | Std: 4.7316e+00 | Min: -1.2991e+01 | Max: 2.1395e+01
cls.predictions.transform.LayerNorm.bias:
Mean: -1.5669e-01 | Std: 4.8085e+00 | Min: -1.8726e+01 | Max: 6.3100e+00
cls.predictions.decoder.weight:
Mean: -5.5245e-02 | Std: 9.7295e-02 | Min: -8.6430e-01 | Max: 5.8028e-01
```

模型参数输出（dim_z=1536）：

```
=== SAVE Step 60000 ===
Last Eval Loss: 2.337440013885498
Hyperparameters:
  dim_z: 1536
  binary_factor_scaling: 5.615
  ternary_factor_scaling: 1.02858
  classifier_amplifier: 164.93668
  regularize_z: 1.62087
  regularize_g: 4.21161
  regularize_h: 0.18722
  learning_rate: 0.041861

model.unary_factors.weight:
  Mean: -1.1305e-01 | Std: 4.5834e+00 | Min: -3.1891e+01 | Max: 2.8695e+01
model.iterator.head_selection.ternary_factor_u:
  Mean: 8.8321e-06 | Std: 2.6467e-02 | Min: -1.3937e-01 | Max: 1.3471e-01
model.iterator.head_selection.ternary_factor_v:
  Mean: 1.3925e-06 | Std: 2.6424e-02 | Min: -1.3414e-01 | Max: 1.4074e-01
model.iterator.topic_modeling.binary_factor:
  Mean: 7.7027e-05 | Std: 2.5827e-02 | Min: -1.4747e-01 | Max: 1.3627e-01
cls.predictions.bias:
  Mean: -6.6317e+00 | Std: 4.8470e+00 | Min: -1.5447e+01 | Max: 4.9901e+00
cls.predictions.transform.dense.weight:
  Mean: 2.5667e-04 | Std: 2.6344e-02 | Min: -1.3449e-01 | Max: 1.2785e-01
cls.predictions.transform.dense.bias:
  Mean: -5.4107e-01 | Std: 4.1034e+00 | Min: -1.6254e+01 | Max: 1.3122e+01
cls.predictions.transform.LayerNorm.weight:
  Mean: 3.0017e+00 | Std: 1.3160e+01 | Min: -3.1163e+01 | Max: 2.5750e+01
cls.predictions.transform.LayerNorm.bias:
  Mean: -1.4619e+00 | Std: 4.2592e+00 | Min: -2.8248e+01 | Max: 9.2310e+00
cls.predictions.decoder.weight:
  Mean: -4.5902e-04 | Std: 7.3497e-03 | Min: -7.6766e-02 | Max: 7.5573e-02
```

参数分类

1. **Input Weights**
   - `model.unary_factors.weight` (Embedding)
   - `cls.predictions.bias` (Decoder Bias, 视作 Input 类处理)
   - `cls.predictions.transform.dense.bias`
   - `cls.predictions.transform.LayerNorm.weight`
   - `cls.predictions.transform.LayerNorm.bias`
2. **Hidden Weights**

- `model.iterator.head_selection.ternary_factor_u`
- `model.iterator.head_selection.ternary_factor_v`
- `model.iterator.topic_modeling.binary_factor`
- `cls.predictions.transform.dense.weight`

3. **Output Weights**
   - `cls.predictions.decoder.weight`

分析

- input weights倍率差(1536的std/96的std):
  - `unary_factors.weight` :0.6682
  - `predictions.bias` :1.1188
  - `predictions.transform.dense.bias` :2.1597
  - `cls.predictions.transform.LayerNorm.weight` :2.7813
  - `cls.predictions.transform.LayerNorm.bias` :0.8858
  - 倍率差基本可认为是$\Theta(1)$
- hidden weights倍率差(96的std/1536的std)：
  - `model.iterator.head_selection.ternary_factor_u` :6.2160
  - `model.iterator.head_selection.ternary_factor_v` :6.3586
  - `model.iterator.topic_modeling.binary_factor` :6.1230
  - `cls.predictions.transform.dense.weight` :6.1843
  - 倍率差基本可认为是$\Theta(1/\sqrt{N})$
- output weights倍率差(96的std/1536的std)：
  - `cls.predictions.decoder.weight` :13.2380
  - 倍率差基本可认为是$\Theta(1/N)$
- 符合预期